

Horizontal Pod Autoscaling of a Containerized Web API: A Practical Comparison of Minikube and Amazon EKS

Julia Meneses Roberto
PSI512 - Cloud Computing Topics, 2026

Abstract- This paper designs, deploys, and evaluates the same CPU-bound FastAPI service under the Kubernetes Horizontal Pod Autoscaler (HPA) in a local Minikube cluster and a managed Amazon Elastic Kubernetes Service (EKS) cluster. The deployment requests 100 millicores per pod, targets 50% average CPU utilization, and permits one to six replicas. Three controlled experiments were executed in each environment. Minikube reached the first scale-up in 49-85 s and returned to one replica in 143-203 s. EKS reacted in 36-55 s and returned in 126-127 s. Every run reached six replicas. The study documents reproducibility, operational differences, security controls, and approximate costs. Results show that HPA behavior is portable, while infrastructure provisioning, public exposure, cost, and operational responsibility differ substantially.

Index Terms- Kubernetes, Horizontal Pod Autoscaler, Amazon EKS, Minikube, cloud computing, elasticity, FastAPI.

I. INTRODUCTION

Elasticity is a defining property of cloud systems: capacity should track demand while avoiding permanent overprovisioning. Kubernetes separates application packaging from scheduling and exposes autoscaling controllers that reconcile observed resource use with a declared target. Horizontal Pod Autoscaler changes the replica count of a scalable workload, providing a practical mechanism for reactive elasticity.

This work implements one reproducible web workload in two environments required by the assignment. The local environment is Minikube running through the Docker driver. The cloud environment is Amazon EKS in us-east-1 with two managed EC2 worker nodes. Keeping the application image, resource requests, HPA target, load generator, and sampling interval equivalent isolates the environment as the principal independent variable.

The contribution is an evidence-backed comparison rather than a configuration-only demonstration. Six experiments record timestamps, current and desired replicas, average CPU utilization, pod counts, node state, HPA events, and final recovery. The repository also includes automated endpoint tests, commented manifests, deployment scripts, a troubleshooting guide, and cleanup automation.

II. BACKGROUND

A. Kubernetes control loops

Kubernetes controllers continuously compare desired and observed state. A Deployment owns ReplicaSets and Pods; a Service supplies stable discovery and routing; and the HPA periodically updates the Deployment scale subresource. This eventual-consistency model means scaling is not instantaneous. Metrics collection, controller synchronization, pod scheduling, image retrieval, readiness checks, and stabilization policies all contribute to response time.

B. HPA utilization semantics

For CPU utilization targets, HPA compares observed CPU consumption with the CPU request declared for each container. A pod requesting 100m and consuming 250m is reported near 250% utilization. The controller estimates a desired replica count proportional to the ratio between current utilization and the 50% target, subject to minimum, maximum, tolerance, and behavior policies.

III. RESEARCH QUESTIONS

The evaluation addresses three questions. RQ1 asks whether the same autoscaling specification can produce observable scale-out and scale-in in both a local and a managed cloud cluster. RQ2 asks how the measured first-expansion and recovery times differ across the two environments. RQ3 asks which operational and economic trade-offs become visible when moving an otherwise unchanged workload from Minikube to EKS.

The working expectation for RQ1 was functional equivalence because HPA, Deployments, and Services expose portable Kubernetes APIs. No directional timing hypothesis was assigned to RQ2: the two platforms have different compute, metrics, and scheduling conditions, and three trials are descriptive rather than inferential. For RQ3, Minikube was expected to minimize setup and direct cost, whereas EKS was expected to provide the more realistic distributed and externally accessible topology.

C. Metrics pipeline

The kubelet reports container resource use through its summary interfaces. Metrics Server aggregates recent CPU and memory measurements and exposes the metrics.k8s.io API. HPA reads that API and the target workload scale, calculates a recommendation, and writes the desired replica count. A missing CPU request makes utilization undefined; a missing or unhealthy Metrics Server produces an unknown HPA target. Therefore numeric output from both kubectl top and kubectl get hpa was an explicit precondition for every run.

D. Reactive elasticity boundary

HPA adds pods, not machines. If the scheduler cannot place the requested pods, replicas remain Pending even though the HPA decision is correct. This distinction motivated the fixed EKS capacity of two nodes and the exclusion of Cluster Autoscaler. It also clarifies that the measured result is application-level horizontal elasticity, not full infrastructure elasticity.

III. SYSTEM ARCHITECTURE

A. Application

The workload is a Python FastAPI service packaged in a non-root Docker image. GET / returns the application identity and version; /health supports readiness and liveness probes; /metrics-info reports the serving pod identity; and /cpu performs bounded SHA-256 work. The CPU route accepts a constrained iteration parameter, allowing sustained processing without external state or unsafe side effects.

Dependencies are pinned. Five automated tests validate the four endpoints, response fields, pod metadata behavior, and invalid CPU parameters. The container runs with a fixed unprivileged UID/GID 10001, disables privilege escalation, uses a read-only root filesystem, drops Linux capabilities, and supplies a writable in-memory /tmp volume.

B. Kubernetes resources

A dedicated namespace isolates all objects. The Deployment begins with one replica and defines 100m CPU request, 500m CPU limit, 96 MiB memory request, and 192 MiB memory limit. HTTP startup, readiness, and liveness probes protect availability. The Service is ClusterIP locally and is patched to LoadBalancer on EKS. The HPA uses autoscaling/v2, one-to-six replicas, a 50% CPU target, immediate scale-up, and a 60 s scale-down stabilization window.

Parameter	Value
Initial/minimum replicas	1
Maximum replicas	6
CPU request / limit	100m / 500m
HPA CPU target	50%
Scale-down window	60 s
Sample interval	approximately 18 s

The in-cluster load generator repeatedly requests /cpu?iterations=400000 using BusyBox wget. Running the producer inside Kubernetes avoids client-network variability and applies pressure through the Service, which distributes requests across ready endpoints.

C. Minikube topology

Minikube used the Docker driver with four virtual CPUs and 6144 MiB memory. Its Metrics Server add-on exported resource metrics. The locally built image was loaded directly into the cluster, so no remote registry or Internet-facing load balancer was required. One logical node hosted the control plane and all workload replicas.

D. Amazon EKS topology

Amazon EKS supplied the managed control plane. A managed node group contained two x86 t3.small instances in separate subnets. The original t3.medium request was rejected by the account's AWS Free Plan, so t3.small was selected and documented as a real experimental constraint. The image was pushed to Amazon ECR with digest sha256:15bc4448550eb2b61c2d847af953e5df31615b7a541f53c2a3ef133b97d47292.

The EKS-managed Metrics Server add-on version v0.9.0-eksbuild.5 reported numeric CPU data. A Kubernetes LoadBalancer Service provisioned a public AWS endpoint. The experiment deliberately omitted Cluster Autoscaler so the study measured pod elasticity only; node capacity remained fixed at two instances.

IV. EXPERIMENTAL METHOD

Each environment was brought to a one-pod baseline. The script recorded context, nodes, all namespaced resources, HPA state, and kubectl top output. It then recreated the load-generator pod, sampled HPA state every 15 s (approximately 18 s including command latency), removed the producer after 180 s, and monitored recovery. Minikube used 300 s recovery; EKS used 240 s because all runs had already converged.

IV. EXPERIMENTAL METHOD (CONT.)

Three independent runs per environment reduce dependence on a single synchronization phase. Scale-up time is the interval from the first load sample to the first sample with desired replicas greater than one. Peak replicas and CPU are maxima across all samples. Recovery time is the interval from the first recovery sample to the first sample with desired replicas equal to one. These definitions are applied identically to both datasets.

The measured timestamps are UTC. Raw samples, HPA descriptions, sorted events, node listings, and before/after resource states were retained locally. Sanitized summary CSV files are versioned. Credentials, kubeconfig data, and secrets are excluded. The public API was smoke-tested through its AWS load-balancer DNS name, while internal health was also verified with a disposable cluster pod.

A. Independent and controlled variables

The independent variable is the runtime environment. Controlled variables include application version, ECR/local image contents, HPA specification, CPU request and limit, load-generator image and command, route parameters, sample schema, and trial count. Infrastructure necessarily differs: Minikube has one local node and EKS has a managed control plane plus two EC2 nodes.

B. Validity considerations

HPA sampling is discrete, so reported response times are upper bounds within roughly one sampling period. The first CPU metric can lag because a new load pod must be scheduled and because Metrics Server aggregates over a window. CPU percentages can reach 500% because the limit is five times the 100m request. Background activity, image caches, network latency, and controller synchronization introduce noise.

The study does not claim statistical generality beyond this workload and account configuration. Three repetitions demonstrate repeatability but are insufficient for broad performance inference. The EKS and Minikube recovery durations differed; however, both windows extended well beyond the first return to one replica, so the measured recovery endpoint is comparable.

V. RESULTS

A. Minikube

Run	Scale-up	Peak CPU	Max pods	Recovery
1	49 s	378%	6	192 s
2	85 s	476%	6	203 s
3	65 s	344%	6	143 s
Mean	66.3 s	399.3%	6	179.3 s

All Minikube runs saturated the six-pod cap. The initial expansion varied by 36 s between the fastest and slowest trials. Once load stopped, utilization fell below target and the configured scale-down stabilization delayed contraction, after which the controller reduced replicas in stages. Every final sample showed one desired and one running application pod.

B. Amazon EKS

Run	Scale-up	Peak CPU	Max pods	Recovery
1	36 s	500%	6	127 s
2	55 s	500%	6	126 s
3	36 s	500%	6	126 s
Mean	42.3 s	500%	6	126.3 s

EKS also reached six replicas in every run. Runs 1 and 3 first requested multiple replicas after 36 s; run 2 required 55 s. Recovery was notably consistent at 126-127 s. Peak observed utilization reached the 500% ceiling in every trial, explaining the immediate request for rapid expansion once metrics reflected load.

C. Distribution and repeatability

Across Minikube trials, scale-up had a 36 s range and recovery had a 60 s range. EKS scale-up had a 19 s range, while recovery differed by only one second. The small EKS recovery spread is consistent with a stable controller and metrics cadence during this session; it is not proof that EKS is universally less variable. Both datasets contain quantization from the sampling interval.

The maximum-replica result is deterministic across all six trials: each sustained load exceeded the capacity represented by six pods at the 50% target. This repeatability validates the load generator for the assignment objective. Peak CPU differs because observed utilization is capped indirectly by the 500m container CPU limit divided by the 100m request. EKS reached this ratio in all trials; Minikube recorded lower peaks in some samples while still requesting the maximum replica count.

D. Representative event sequence

Phase	Observed controller behavior
Baseline	1 ready pod; CPU below 50%
Load begins	CPU metric rises after aggregation delay
First expansion	desired replicas changes to 3
Peak	desired/current replicas reach 6
Load removed	CPU decays as requests drain
Stabilization	six pods retained temporarily
Contraction	6 to 3/2 to 1 replicas

Kubernetes events corroborated the sampled CSVs by recording SuccessfulRescale actions and Deployment replica changes. Final-state captures showed the load pod absent, one application pod ready, and desired replicas equal to one. Thus the conclusion does not depend on a single command or screenshot.

VI. COMPARATIVE ANALYSIS

Metric	Minikube mean	EKS mean	Difference
First scale-up	66.3 s	42.3 s	EKS -24.0 s
Return to one	179.3 s	126.3 s	EKS -53.0 s
Maximum replicas	6	6	equal
Peak CPU	399.3%	500%	+100.7 pp EKS

The cloud environment reacted faster in these trials, but the difference must not be attributed to EKS alone. Controller phase alignment, metrics windows, node CPU scheduling, and load distribution can change observed timing. The important portable result is functional equivalence: the identical HPA policy expanded and contracted successfully in both environments.

A. Scaling dynamics

EKS commonly moved from one to three desired replicas and then to six on the next observation. Minikube followed the same qualitative pattern with greater timing variation. High per-pod utilization persisted during load because the generator supplied enough parallel work to saturate each new replica. Hitting maxReplicas prevented further growth and bounded resource consumption.

Scale-down is intentionally slower than scale-up. Abrupt contraction during transient dips can create oscillation and harm availability. The 60 s stabilization window retained recent recommendations, and a 50% per 30 s downscale policy limited contraction. Observed recovery therefore combines workload drain, metrics decay, stabilization, and staged termination.

B. Operational effort

Minikube offered the shortest development loop: one command started the cluster, an add-on enabled metrics, and the local image was loaded without a registry. It was also cost-free beyond the host computer. Its limitations are equally clear: one machine is not highly available, public ingress is artificial, and capacity is constrained by developer resources.

EKS required IAM authorization, eksctl, CloudFormation, ECR authentication, a managed node group, Metrics Server, a public load balancer, and explicit cleanup. Provisioning took substantially longer than deploying the workload. In exchange, EKS provided a managed API server, AWS networking integration, managed worker lifecycle, and a realistic multi-node topology.

C. Reliability and capacity

The two EKS nodes allowed the scheduler to distribute replicas across failure domains at the worker level. The Deployment used rolling updates and readiness probes, so Services excluded unready endpoints. Minikube validated Kubernetes semantics but remained a single-host laboratory. Neither environment used node autoscaling; a production system would coordinate HPA with node capacity and disruption budgets.

VII. COST ANALYSIS

The local experiment incurred no direct cloud charge. Its economic cost is developer hardware, electricity, and time. For EKS, the principal metered components are the control plane, two EC2 instances, an Elastic Load Balancer, EBS root volumes, and minor ECR storage/data transfer. Prices vary by date, region, account plan, taxes, and free-plan credits; therefore the following values are planning estimates rather than billing evidence.

Component	Illustrative rate	3 h experiment
EKS control plane	about \$0.10/h	\$0.30

Component	Illustrative rate	3 h experiment
2 x t3.small	about \$0.0416/h total	\$0.12
Load balancer	about \$0.0225/h + usage	\$0.07+
EBS/ECR	small prorated amount	<\$0.02
Approximate total	before credits/tax	about \$0.51

VII. COST ANALYSIS (CONT.)

The estimate assumes approximately three hours of live infrastructure. The actual session may differ, and AWS Free Plan eligibility can offset some instance charges. EKS control-plane and load-balancer charges may still apply. The safest practice is to automate teardown and verify deletion of the Service, node groups, cluster stacks, and ECR repository after evidence is secured.

VIII. SECURITY AND REPRODUCIBILITY

A. Container and workload security

The image uses a small Python base, pinned dependencies, a non-root account, and an explicit command. Kubernetes securityContext settings enforce non-root execution, disable privilege escalation, drop capabilities, and mark the root filesystem read-only. CPU and memory limits constrain accidental or malicious resource consumption. Health probes separate process liveness from traffic readiness.

B. Cloud security

AWS actions were executed in an authenticated console session without exporting credentials into the repository. ECR stored only the public project image. The LoadBalancer exposed HTTP solely for the assignment test and was scheduled for deletion afterward. A production design would add TLS, a restricted ingress layer, web-application protections, private worker networking, least-privilege IAM roles, image scanning, and centralized audit logs.

C. Artifact hygiene

The repository ignores private keys, certificates, environment files, kubeconfigs, raw evidence, caches, and generated secrets. Raw cloud outputs remain local until reviewed because node names, account identifiers, and public addresses may appear even when no credential is present. Sanitized CSV summaries preserve scientific values without disclosing secrets.

D. Reproduction procedure

A new operator installs Python, Docker, kubectl, Minikube, AWS CLI, and eksctl; runs the test suite; builds the image; and applies the Kustomize overlay. On EKS, the operator creates the cluster from eks/cluster.yaml, pushes the same tag to ECR, installs the managed Metrics Server add-on, substitutes the ECR URI in a temporary manifest copy, and waits for numeric metrics before executing the three trials.

Every source file, Docker instruction, manifest, and automation script includes comments explaining intent or safety constraints. Generated evidence directories are deterministic: evidence/raw/<environment>/run-<n>. Each trial captures context, baseline, node metrics, pod metrics, samples, events, HPA description, and final state. The deployment guide also records smoke tests, screenshot requirements, troubleshooting, and cleanup.

IX. LIMITATIONS AND FUTURE WORK

The CPU endpoint represents compute-bound demand rather than a complete application mix. No latency percentile or request throughput was collected, so autoscaling effectiveness is inferred from resource response and replica state, not user-perceived service-level objectives. Future work should pair Prometheus request metrics with CPU, define latency/error objectives, and use a controlled load tool that reports arrival rate and completed requests.

The two-node EKS group was fixed. If six replicas plus system pods exceeded capacity, HPA could request pods that remain Pending. Adding Cluster Autoscaler or Karpenter would allow joint pod/node elasticity, but it would introduce another response loop and additional

variables. Repeating the study across instance families, warm/cold image caches, and longer steady-state loads would improve external validity.

The repository intentionally stops at the intermediate assignment boundary. After submission is confirmed, the preserved tag can be extended with a simulated IoT device, AWS IoT Core, MQTT over mutual TLS, Lambda, DynamoDB, Device Shadow, and API queries while retaining HPA. That extension is not part of the present results.

X. TROUBLESHOOTING OBSERVATIONS

A. Instance-type eligibility

The first EKS node-group attempt used t3.medium as originally planned. Auto Scaling rejected every launch because that type was not eligible under the account's AWS Free Plan. The failed managed node group was deleted and replaced with workers-small using two t3.small instances. This incident demonstrates why cloud instructions must include account-plan and quota checks rather than assuming that a commonly available instance type is selectable.

B. Archive permissions

The project bundle was created on Windows and uploaded to CloudShell. Directory execute bits in the overlay tree were absent after extraction, preventing traversal even though files were readable. Restoring mode 755 on directories resolved the issue. The repository procedure now treats cross-platform archive permissions as a deployment preflight concern.

C. Metrics readiness

Immediately after Deployment creation, the HPA target appeared as unknown because its first metric had not yet arrived. Waiting for the managed Metrics Server add-on to become ACTIVE and verifying kubectl top nodes prevented a false failure diagnosis. Experiments began only after the HPA displayed numeric utilization at the one-pod baseline.

D. Load balancer choice

Annotations requesting an external Network Load Balancer would require the optional AWS Load Balancer Controller in this cluster design. The overlay therefore uses the standard LoadBalancer Service without controller-specific annotations. The EKS cloud integration provisioned a public endpoint suitable for the short-lived HTTP validation, and cleanup removes it before cluster deletion.

XI. PRACTICAL LESSONS

Four practices improved confidence in the result: test the application before containerization; validate metrics before interpreting HPA state; sample controller state rather than relying on a single peak screenshot; and retain teardown until every raw file and rendered article page has been checked. These practices separate implementation errors from expected control-loop delays and protect evidence from premature deletion.

XII. CONCLUSION

This project implemented a complete CPU-based Kubernetes HPA experiment on Minikube and Amazon EKS using the same tested FastAPI image and commented manifests. Six real executions demonstrated scale-out from one to six replicas and successful convergence back to one replica. Minikube averaged 66.3 s to first scale-up and 179.3 s to return to one; EKS averaged 42.3 s and 126.3 s respectively.

The results support three conclusions. First, declarative Kubernetes autoscaling policies are portable across local and managed clusters when resource requests and Metrics Server are correctly configured. Second, timing is influenced by metrics and control-loop phases, so multiple runs and explicit measurement definitions are necessary. Third, managed cloud operation adds realistic networking and resilience but also provisioning complexity, account constraints, and direct cost.

For teaching and iteration, Minikube is the more economical and convenient environment. For managed multi-node behavior and public integration, EKS is more representative. They are complementary rather than interchangeable: local validation catches application and manifest defects early, while EKS validates cloud-specific image distribution, node management, metrics integration, and external service exposure.

ACKNOWLEDGMENT

The author thanks the PSI512 teaching staff for the assignment specification and the official Kubernetes and AWS documentation teams for the operational references.

REFERENCES

- [1] Kubernetes Authors, 'Horizontal Pod Autoscaling,' Kubernetes Documentation, accessed Aug. 16, 2026. <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>
- [2] Kubernetes Authors, 'Horizontal Pod Autoscaling Walkthrough,' accessed Aug. 16, 2026. <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/>
- [3] Amazon Web Services, 'Scale pod deployments with Horizontal Pod Autoscaler,' Amazon EKS User Guide, accessed Aug. 16, 2026. <https://docs.aws.amazon.com/eks/latest/userguide/horizontal-pod-autoscaler.html>
- [4] Amazon Web Services, 'Amazon EKS pricing,' accessed Aug. 16, 2026. <https://aws.amazon.com/eks/pricing/>
- [5] Amazon Web Services, 'Amazon EC2 On-Demand Pricing,' accessed Aug. 16, 2026. <https://aws.amazon.com/ec2/pricing/on-demand/>
- [6] Amazon Web Services, 'Elastic Load Balancing pricing,' accessed Aug. 16, 2026. <https://aws.amazon.com/elasticloadbalancing/pricing/>
- [7] D. Bernstein, 'Containers and Cloud: From LXC to Docker to Kubernetes,' IEEE Cloud Computing, vol. 1, no. 3, pp. 81-84, 2014.
- [8] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, 'Borg, Omega, and Kubernetes,' Communications of the ACM, vol. 59, no. 5, pp. 50-57, 2016.
- [9] Kubernetes Authors, 'Resource Management for Pods and Containers,' accessed Aug. 16, 2026. <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>
- [10] Minikube Authors, 'Minikube Documentation,' accessed Aug. 16, 2026. <https://minikube.sigs.k8s.io/docs/>

APPENDIX A. ACCEPTANCE EVIDENCE

Criterion	Minikube	Amazon EKS
API healthy	Passed	Passed
Numeric Metrics Server data	Passed	Passed
HPA expanded in 3/3 runs	Passed	Passed
Maximum six pods observed	Passed	Passed
Return to one in 3/3 runs	Passed	Passed
Final desired replicas	1	1

Repository evidence includes the six sanitized summary rows, three raw sample sets per environment retained locally, HPA descriptions, sorted events, node listings, before/after metrics, test output, and the image digest. The ta1-submission tag is created only after the final PDF and repository state are validated.